

Infranex BT

The Complete Mining Workflow

A field manual for running Bittensor miners on the Infranex BT platform. It walks the full path in exact order — researching subnets, renting GPU capacity, connecting the node daemon, testing against the Judge Lab, and applying fixes to live machines — with every click, every checkpoint, and the recovery steps for when something goes wrong.



Infranex BT Mining Operations

Platform guide · 15 views · Judge Lab closed loop · DevOps Engine

The Golden Path — Workflow at a Glance

Infranex BT is a Bittensor mining operations platform: it watches the TAO chain, rents GPU capacity from providers, drives your miners through a node daemon, and continuously tests each miner against a simulated judge to tell you exactly what to fix. The whole product is built around one repeating loop, and every view in the sidebar exists to serve one step of that loop. This chapter gives you the map; the rest of the handbook walks each step in full detail, in the exact order you should perform them.

The single most important ordering rule in this handbook is this: **the daemon must be online before you run the Judge Lab's Apply step**. Mining a judge profile runs on the platform server, so it works even with no GPU attached — but the moment you click **Apply**, the platform pushes a config change to a real machine through that machine's daemon. No daemon, no push. Follow the phases in order and this can never bite you.

1

Pick your subnet

SUBNETS · OPPORTUNITIES

Sync live chain data, compare ROI, deregistration risk and free seats, and choose the netuid you want to mine.

Output: a chosen subnet (netuid)

2

Deploy the GPU

GPU CATALOG → DEPLOY WIZARD

One guided wizard: choose subnet → check requirements ↗ pick a GPU offer ↗ rent and auto-install ↗ register the hotkey.

Output: a deployment card in state "started"

3

Connect the node daemon

DEPLOYMENTS → INSTALL DAEMON

Paste one setup command on the GPU pod (SSH, or fully automated via the provider start script). The daemon is the platform's hands on your machine.

Output: daemon chip shows Online

4

Run the Judge Lab

JUDGE LAB

Select the same subnet, re-mine the profile once, then run the simulation. You get a verdict, a composite score, and per-dimension recommendations.

Output: verdict + ranked fix list

5

Apply the fixes

JUDGE LAB → APPLY

One click per fix: the platform maps the recommendation to a config change, snapshots the current revision, and pushes it to your miner via the daemon.

Output: tuned miner + rollback snapshot

6

Operate — then leave it alone

MONITORING · DEVOPS ENGINE

Monitoring and the 14 DevOps triggers watch the fleet for you. You touch Judge Lab again only when an alert fires, the subnet changes upstream, or the weekly review comes around.

Output: sustained earnings, early warnings

The one-liner to remember: Subnet → GPU → daemon online → Judge → Apply → then it is hands-off until an alert tells you to re-judge. If you ever wonder “when do I run the Judge Lab?” — the answer is **after** the daemon chip turns green, and **again** whenever Monitoring or the DevOps Engine says something drifted.

PHASE 0 · ONE-TIME SETUP

Before You Touch Any Subnet

Three things must be in place before the mining loop can run: an account on the platform, a provider API key so the platform can rent GPUs on your behalf, and a Bittensor wallet for on-chain registration. Each is a five-minute job, and you only do them once. If you inherited a working install, skim this chapter anyway — it tells you which keys the later phases depend on, so you know what breaks if one is missing.

1

Log in to the platform

Open the web app and enter your **User ID** and **Access Code**. The code is stored hashed and encrypted; there is no recovery flow, so keep the original from your operator. Successful login lands you on the Dashboard with live chain data (block height, TAO price, subnet count) already streaming.

2

Connect a provider key

Go to **GPU Catalog** and click **Connect provider key**. Paste a RunPod API key (and a Vast.ai key if you plan to compare both). Until a key is configured the offers list is empty — the header will say `no live offers yet` as a hint. With a key present you get live pricing: roughly 45 GPU types with real-time hourly rates.

3

Have your wallet ready

The Deploy Wizard's last step registers a hotkey on-chain, which needs your **coldkey mnemonic / seed** (or an existing hotkey pair). The platform walks you through registration in Phase 2 step 5 — you only need the seed phrase available, funded with a small TAO balance for the registration fee if the subnet charges one.

CHECKPOINT — YOU ARE READY WHEN

The Dashboard renders live chain numbers, GPU Catalog shows live offers from your provider, and your wallet seed is at hand. Nothing here involves a GPU machine yet.

PHASE 1 · RESEARCH

Pick Your Subnet

A subnet is a market: it defines what kind of miner you must run, how seats are priced, and how brutal the competition is. Choosing well matters more than any later tuning — a good miner on a bad subnet loses money on electricity it never earns back. The **Subnets** view (with **Opportunities** as its aggressive cousin) gives you the live ranking; your job here is to shortlist one netuid and note its requirements.

1 Sync the chain data

Open **Subnets** and click **Sync all**. The table refreshes from the chain RPC every ~30 seconds on its own, but an explicit sync guarantees the seat counts and emission numbers you are about to judge are current, not cached.

2 Filter to active subnets

Use the **active-only** filter chips to hide dead or speculative subnets. There is no point evaluating a subnet whose emission is zero or whose registration is effectively closed — the filter does that pre-work for you.

3 Compare the three decision numbers

For each candidate read: **ROI score** (estimated earnings against current cost), **dereg risk** (how likely underperformers are purged — high risk means high churn and high skill floor), and **free seats** (whether registration can even succeed). The Opportunities view adds cross-subnet arbitrage if several subnets look close.

4 Choose and write it down

Pick one **netuid** and note it together with the subnet's minimum VRAM from the requirements shown later — you will type the netuid into the Judge Lab and the Deploy Wizard, so having it in front of you avoids mistakes.

CHECKPOINT — YOU ARE DONE WHEN

You have one netuid chosen and a rough expectation of its VRAM floor. Do not over-research: the Judge Lab in Phase 4 will verify the economics against your actual machine anyway.

PHASE 2 · DEPLOY

Deploy the GPU — the Five-Step Wizard

Everything in this phase happens inside one dialog: the **Deploy Wizard**. It replaces what used to be scattered across three screens — nothing is out of order or missing, because rent, install, and registration flow end-to-end. You can enter it two ways: from **GPU Catalog**, press **Provision** on the offer you like (the GPU arrives preselected), or from **Subnets / Opportunities**, press start-mining on the card (the subnet arrives preselected and the wizard

opens directly on the requirements step). Either way, the five steps below run in the same order.

1 Step 1 — Choose the subnet

A live chain list with seat verdicts per subnet. Pick the netuid you chose in Phase 1, or let a preselection stand. The wizard shows whether registration is realistic before it lets you spend money.

2 Step 2 — Check the requirements

The wizard fetches the subnet's real requirements profile and displays the minimum VRAM and model expectations. **Do not skip this check** — renting a GPU below the VRAM floor is the single most common wasted-rent mistake, and the recommendation card on GPU Catalog repeats the same warning for H100s.

3 Step 3 — Pick the GPU offer

Live offers from your configured providers, filterable by provider and max price, sortable by hourly price or VRAM. Match or exceed the VRAM floor from step 2, sanity-check the hourly rate against the subnet's expected emission, and select the offer.

4 Step 4 — Rent and auto-install

Confirming the rent creates the deployment and starts a real ten-step install runner with a live log. You can literally watch each step complete; the lifecycle auto-advances while you watch, and when the deployment status flips to `started` the wizard moves itself to the final step.

5 Step 5 — Hotkey & run

The wallet registration hand-off. Follow the registration wizard (coldkey seed → hotkey creation → on-chain register) to put a live key on the subnet. When it completes, close the dialog — a **deployment card** now exists in Deployments with monthly cost, estimated revenue, and lifecycle progress.

CHECKPOINT — YOU ARE DONE WHEN

Deployments lists your new card in state **started** with cost/revenue figures populated. The machine exists and the miner stack is installed — but the platform still cannot reach inside it. That is the daemon's job, next.

Note on mock deployments: if you were experimenting without a real provider key, the platform can create a simulated deployment that ticks through the same lifecycle. Mocks are clearly labeled, but remember they cannot serve real traffic and should not be mixed into a production fleet's dashboards.

PHASE 3 · CONNECT

Connect the Node Daemon

The daemon is a tiny agent that runs on your GPU pod and polls the platform for work. Everything downstream — Judge Lab's **Apply**, DevOps automated responses, config edits from the deployment card — travels through it. Installing it is the one genuinely manual moment in the whole workflow, and even that is a single copy-paste. The dialog itself summar-

izes the contract: *SSH as root → paste one command → wait about 60 seconds → the chip turns Online.*

1 Open the install dialog

In **Deployments**, find your deployment card and click **Install daemon**. The dialog shows the pre-built setup command with the daemon identity and secret already embedded for *this specific deployment* — never reuse one deployment's command on another machine.

2 Copy the setup command

Click **Copy setup command (auto-start service)**. The command is self-contained and idempotent: it writes the daemon, installs an auto-start service, and cleans up any older install. On a VM or bare metal it registers a `systemd` unit (survives reboots, restarts on crash); inside containers like RunPod or Vast it falls back to a `watchdog` loop that relaunches the daemon every 15 seconds.

3 Run it on the GPU pod

SSH into the pod as **root** and paste the command into the shell. The pod prints which launcher it chose — `launcher=systemd` or `launcher=watchdog` — so you always know what is keeping it alive.

4 Watch for Online

Within about a minute the deployment card's daemon chip flips to **Online** (the platform shows a "last seen" age in seconds once heartbeats arrive). If it stays offline after several minutes, jump to the troubleshooting chapter — the usual causes are a wrong platform URL inside the pod or an egress firewall.

5 Optional — automate it away

To never SSH again: paste the same setup command into your provider's start script instead (**RunPod**: template → Start script; **Vast.ai**: on-start script; any cloud: cloud-init user-data). Every fresh boot then installs or refreshes the daemon automatically. Use **one template per deployment** — the embedded identity and secret are deployment-bound, so sharing a template across machines would give them one identity.

CHECKPOINT — YOU ARE DONE WHEN

The daemon chip on the deployment card is green **Online** with a fresh "last seen". From this moment, Judge Lab Apply has somewhere real to push to.

Updating later: re-paste the setup command any time — it is idempotent, so re-running it upgrades the daemon in place (`systemd` restarts the service; the `watchdog` replaces the old process). The dialog also carries an `uninstall` command if you ever decommission the pod.

PHASE 4 · VERIFY

Judge Lab — Test, Read the Verdict, Apply

The Judge Lab is the platform's dress rehearsal: it mines a profile of what your target subnet's validator actually rewards, simulates your miner against it, and returns a verdict with per-dimension scores — quality, resource fit, throughput, speed, price, availability. Each weak dimension comes with a concrete recommendation, and each recommendation carries

an **Apply** button that pushes the fix to your live machine. This is the loop you will re-enter many times over the miner's life, so learn it once here.

1 Select the subnet

Open **Judge Lab** and pick the **same netuid** your deployment mines. The judge's recommendations are subnet-specific — a fix computed for one subnet's judge may be wrong for another.

2 Mine the profile (first time only)

If no profile exists yet the panel says so — press **Re-mine profile** and wait while the platform extracts the subnet's reward rules. This runs server-side and does not need your GPU. Re-mine again later whenever the subnet's upstream code changes.

3 Run the simulation

Press **Run simulation**. While it scores, the button reads `Scoring against judge...`. When it finishes you get a **Verdict** panel: overall verdict badge, composite score, per-dimension scores, and a ranked list of the highest-leverage fixes.

4 Apply the fixes you accept

Click **Apply** on a recommendation. A picker lists your deployments — same-subnet ones sorted first — with a preview of the computed change (for price fixes, the exact target USD per M-token). Confirm, and the platform maps the fix to a recipe (for example: quality → fp8 weights plus self-verify; throughput → vLLM; price → the computed ask price), snapshots the current revision, then pushes `apply_config` to the daemon. A result note confirms the push; mock deployments receive a simulated tick instead.

5 Know that verdicts survive

The **Recent runs** list keeps every simulation. Click any row to restore that verdict and its Apply buttons — nothing is lost by navigating away or by the page reloading. Availability fixes are deliberately manual; the judge will say so instead of pretending to automate them.

CHECKPOINT — YOU ARE DONE WHEN

At least one fix is applied, the deployment card's config tab shows the changed environment, and a new revision snapshot exists for rollback. The miner now runs the judge-approved configuration.

When should you run the Judge Lab? The table below is the complete answer — first time is immediately after the daemon comes online, and every later run is triggered by one of four events. If none of these happened, there is nothing to re-judge; spend the time on Monitoring instead.

Moment	Why
Right after the daemon turns Online	First configuration pass. Apply needs a live deployment, so this is the earliest useful moment — profile mining alone can run earlier, but it produces nothing actionable without a connected miner.
A DevOps Engine alert fires	GPU HEALTH, SUBNET DRIFT, BENCH REGRESS and friends tell you reality moved; re-run the simulation and apply fresh fixes.
The subnet changes upstream	After an upstream release or rule change, re-mine the profile first, then re-run — the old profile is judging an outdated rulebook.
Weekly review	A fixed weekly slot catches slow drift that alerts missed. Fifteen minutes: re-run, compare composite to last week, apply if it dropped.

Operate — Monitor, Automate, Re-Judge

Once a fix is applied, the system is designed to run without you. **Monitoring** is the health picture: inference scores, service latency, GPU health, query volume, and earnings against cost, all updating continuously. **The DevOps Engine** is the action layer on top: fourteen trigger types watch the same telemetry and respond automatically — restarting a sick miner, re-syncing after an upstream change, scaling capacity, or escalating to you when a decision is genuinely yours. Your daily job shrinks to reading one screen and letting the engine do the rest.

Monitoring — what to glance at daily

- **Composite / inference score** — the miner's actual standing; a slow slide means the judge profile aged.
- **Daemon heartbeats** — "last seen" age should stay in seconds, never minutes.
- **Cost vs estimated revenue** — the deployment card's two headline numbers must stay in healthy proportion.
- **Alert stream** — anything the DevOps Engine escalated to a human.

DevOps Engine — the 14 triggers

GPU HEALTH · SUBNET DRIFT · DEREG-RISK · ARBITRAGE · RUNTIME OPT · PROBE FAIL · SERVICE LATENCY · QUERY DROUGHT · ESCALATION · UPSTREAM · BENCH REGRESS · RE-SYNC · SCALE · KILL. Each trigger documents what it watches and what it does; treat UPSTREAM and BENCH REGRESS as your "go re-judge" signals, and KILL as the engine's last-resort circuit breaker for a runaway deployment.

Reading alerts correctly: an alert names a symptom, not a culprit. A data-source hiccup (chain RPC, price feed) can look identical to a broken miner in a dashboard — before touching the machine, check the System & Errors view to confirm the platform's own feeds are healthy. This one habit prevents the most common false-alarm fire drills.

RECOVERY

If Something Goes Wrong

Failures have a natural diagnosis order: rule out the platform itself first, then the pipeline, then the machine, then the config. Work top to bottom — each step either finds the problem or definitively excludes a layer, which is exactly what you want when an alert is screaming and TAO is burning. The single most valuable habit is resisting the urge to redeploy from scratch; almost nothing in this stack needs that.

1

Check the platform's own health

Open **System & Errors**. If chain RPC, price feeds, or provider APIs show failures there, the "miner problem" in Monitoring may be a data-source outage wearing a costume. Wait it out or re-sync before deeper digging.

- 2 **Read Monitoring for scope**
One miner sick or the whole fleet? One subnet or all? Scope tells you where to look next: a single deployment points at the machine or its config; everything-at-once points at the platform or a global data source.
- 3 **Inspect the deployment card**
Open the card's **logs** and **config** tabs. The logs show the install runner and daemon activity; the config tab shows the exact environment the miner is running — compare it against what the last Judge apply claimed to set.
- 4 **Roll back with Revisions**
If the card got sick right after an apply, open the **Revisions** dialog and restore the previous snapshot. Every apply creates one precisely so this step is one click, never archaeology.
- 5 **Re-install the daemon if it vanished**
Daemon chip gray? Re-paste the same setup command from **Install daemon** — it is idempotent and will cleanly reinstall or upgrade the agent. On spot/ephemeral machines the pod may have been evicted entirely; redeploy from Phase 2.

Symptom	Likely cause	First move
Daemon chip gray / offline	Pod rebooted without auto-start, or spot eviction	Re-paste setup command; if SSH fails, the pod is gone — redeploy
Judge Apply errors	No live deployment selected, or daemon offline	Verify daemon Online, retry; check the result note for the push ladder's message
GPU Catalog: no offers	Provider key missing, expired, or out of credit	Reconnect the provider key in GPU Catalog
Sudden composite drop	Upstream change or data-source outage	System & Errors first, then Re-mine profile + re-run judge
Config change didn't stick	Apply pushed but daemon applied an older revision	Check Revisions; re-apply the fix and confirm the note

CADENCE

Operating Rhythms & Quick Reference

The platform rewards a calm operator more than a busy one. The rhythms below are the entire ongoing workload once your first miner is tuned: a one-time setup measured in minutes, a daily glance of roughly five, and a weekly fifteen-minute review. Everything else is event-driven — the DevOps Engine pings you when reality changes and stays silent when it hasn't.

Rhythm	Time	What you actually do
First-time setup	~30 minutes, once	Phase 0 keys → pick subnet → run the Deploy Wizard → daemon online → judge → apply the top fixes.

Daily	~5 minutes	Glance at Monitoring: composite, daemon ages, cost vs revenue, escalations. Touch nothing that is green.
Weekly	~15 minutes	Re-run Judge Lab, compare the composite with last week, re-mine the profile if upstream changed, apply drifts. Skim System & Errors for recurring warnings.
On alert	as needed	Follow the recovery order: System & Errors → Monitoring scope → card logs/config → Revisions rollback → daemon re-paste.

Quick reference — the whole loop, with exact button names:

Step	Where	Action
Pick	Subnets	Sync all → active-only filter → choose netuid (ROI, dereg risk, seats)
Deploy	GPU Catalog	Connect provider key (once) → Provision → wizard: Subnet → Requirements → GPU → Install → Hotkey & run
Connect	Deployments	Install daemon → Copy setup command → paste on pod (SSH or provider start script) → chip Online
Judge	Judge Lab	Select subnet → Re-mine profile (first time / upstream change) → Run simulation → read verdict
Apply	Judge Lab	Apply on a recommendation → pick deployment → confirm → snapshot + push
Operate	Monitoring · DevOps	Daily glance; let the 14 triggers act; re-judge on UPSTREAM / BENCH REGRESS / weekly
Recover	System & Errors	Platform health first, then card logs, then Revisions rollback

Strengths and cautions, honestly stated: the platform's strongest assets are the honest transmission labeling (mock vs live is never blurred), the revision snapshots behind every apply, and the closed Judge→Apply loop. Its sharp edges: preview URLs can leak into copied configs, mock deployments can pollute fleet views if you experiment in the same workspace, spot instances can be evicted mid-run (prefer persistent pods for long-lived miners), and a data-source outage can masquerade as a miner failure — always check System & Errors before surgery.

THE WHOLE HANDBOOK IN ONE LINE

Mine. Judge. Apply. Repeat.

The golden path never changes: pick a subnet worth mining, put a real GPU under it, connect the daemon so the platform has hands on the machine, let the Judge Lab find the gap between your miner and the validator's expectations, and apply the fixes with one click. After that, Monitoring and the DevOps Engine keep watch — and you only return when an alert, an upstream change, or the weekly review calls you back.

SUBNET → GPU → DAEMON ONLINE → JUDGE → APPLY → OPERATE